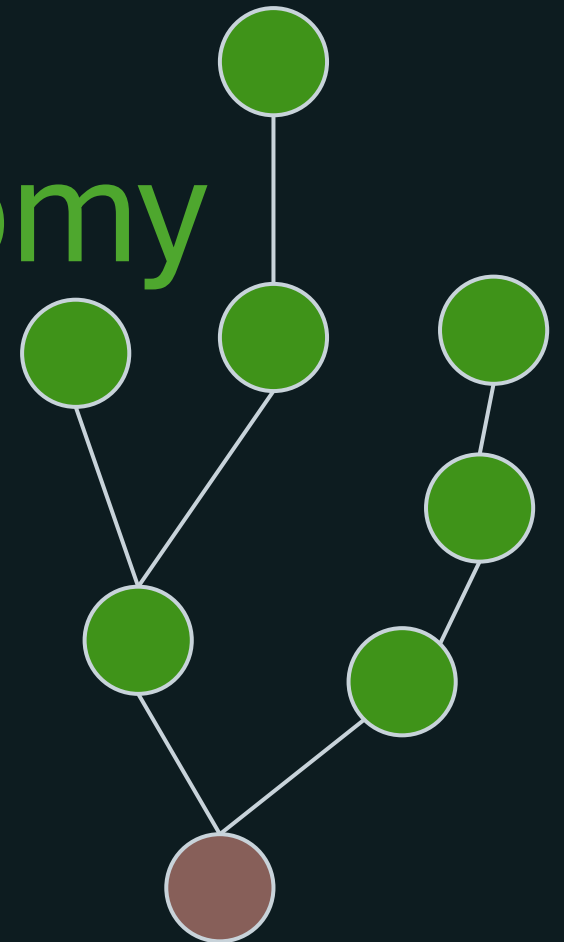
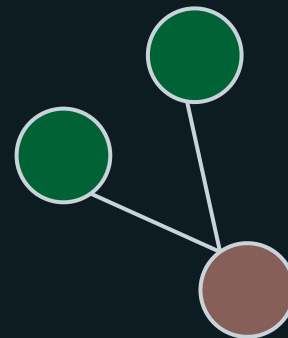
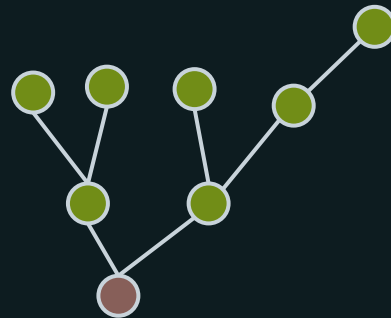
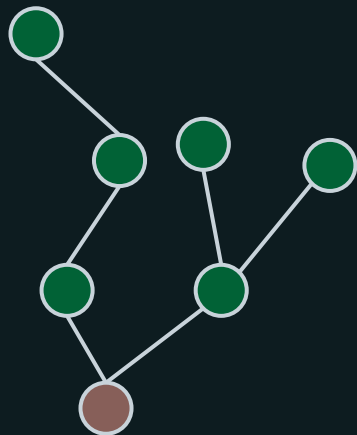
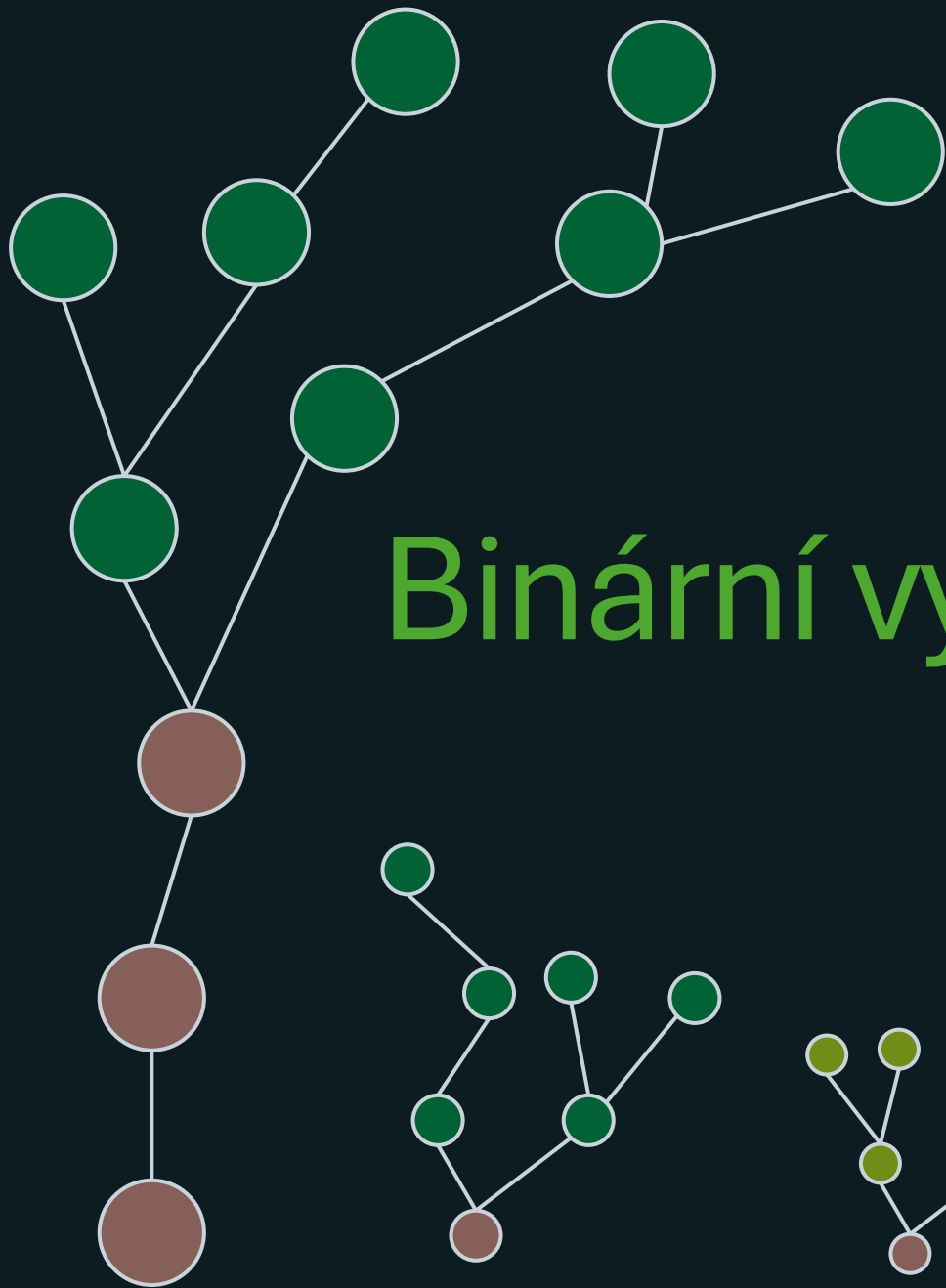


Binární vyhledávací stromy

Binary Search Tree



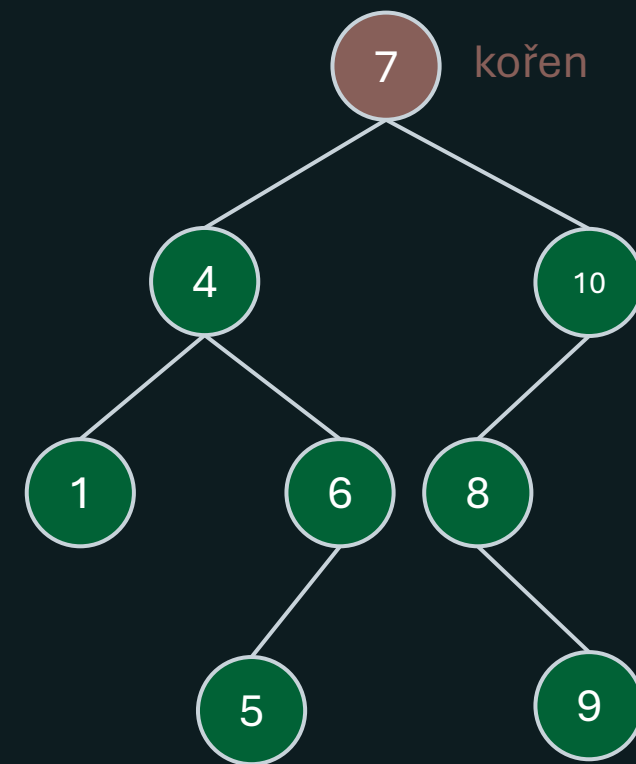
Definice a značení

- **Strom** – spojitý graf, který neobsahuje cyklus
- **Binární** – je zakořeněný a každý jeho vrchol má 0-2 syny: rozlišujeme *levý* a *pravý*
- **Vyhledávací** – aby tato datová struktura sloužila k rychlému vyhledávání, je uspořádání stromu je ovlivněno podmínkami:

Každému vrcholu v přiřadíme klíč $k(v)$. Pak platí pro všechny v v BVS:

- Kdykoli $a \in L(v)$, pak $k(a) < k(v)$
($L(v)$ je **levý** podstrom vrcholu v)
- Kdykoli $a \in R(v)$, pak $k(a) > k(v)$
($R(v)$ je **pravý** podstrom vrcholu v)

- **Úplný** – všechny úrovně stromu zcela zaplněny uzly (každý má 2 potomky), s možnou výjimkou poslední úrovně



Hloubka stromu $h(\text{kořen}) = 4$

Základní operace

Vypiš strom (*Show*)

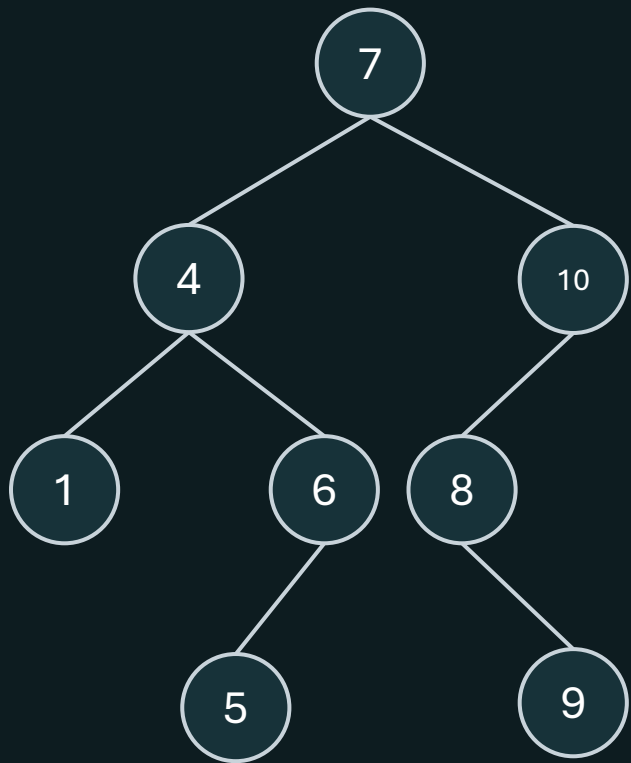
Najdi prvek (*Find*)

Najdi minimum (*FindMin*)

Vlož prvek (*Insert*)

Smaž prvek (*Remove*)

Základní operace – Vypiš strom (*Show*)



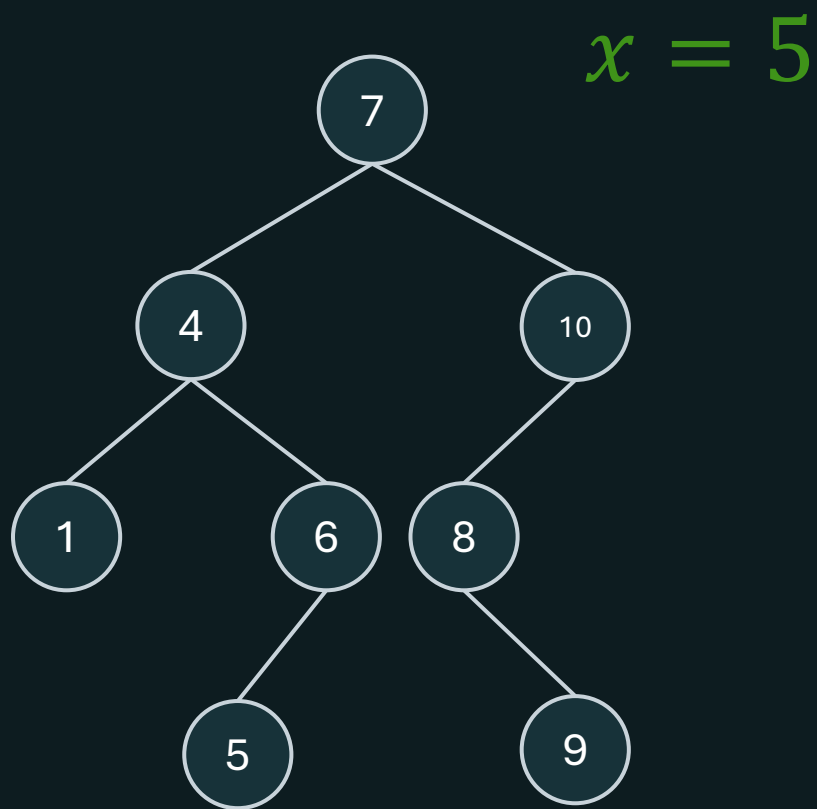
Procedura BvS**SHOW** (uspořádaný výpis BVS)

Vstup: Kořen BVS v

1. Pokud $v = \emptyset$, jedná se o prázdný strom a hned skončíme.
2. Zavoláme BvS**SHOW**($\ell(v)$).
3. Vypíšeme klíč uložený ve vrcholu v .
4. Zavoláme BvS**SHOW**($r(v)$).

1 4 5 6 7 8 9 10

Základní operace – Najdi prvek (*Find*)



Vrátíme tento vrchol

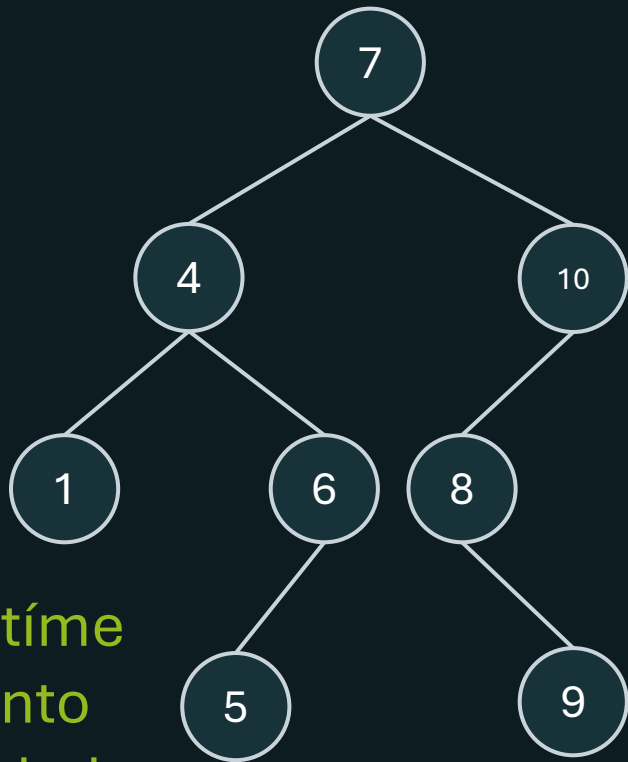
Procedura BVSFIND (hledání v BVS)

Vstup: Kořen BVS v , hledaný klíč x

1. Pokud $v = \emptyset$, vrátíme $\emptyset$.
2. Pokud $x = k(v)$, vrátíme v .
3. Pokud $x < k(v)$, vrátíme BVSFIND($\ell(v)$, x).
4. Pokud $x > k(v)$, vrátíme BVSFIND($r(v)$, x).

Výstup: Vrchol s klíčem x , anebo $\emptyset$

Základní operace – Najdi minimum (*FindMin*)



Vrátíme
tento
vrchol

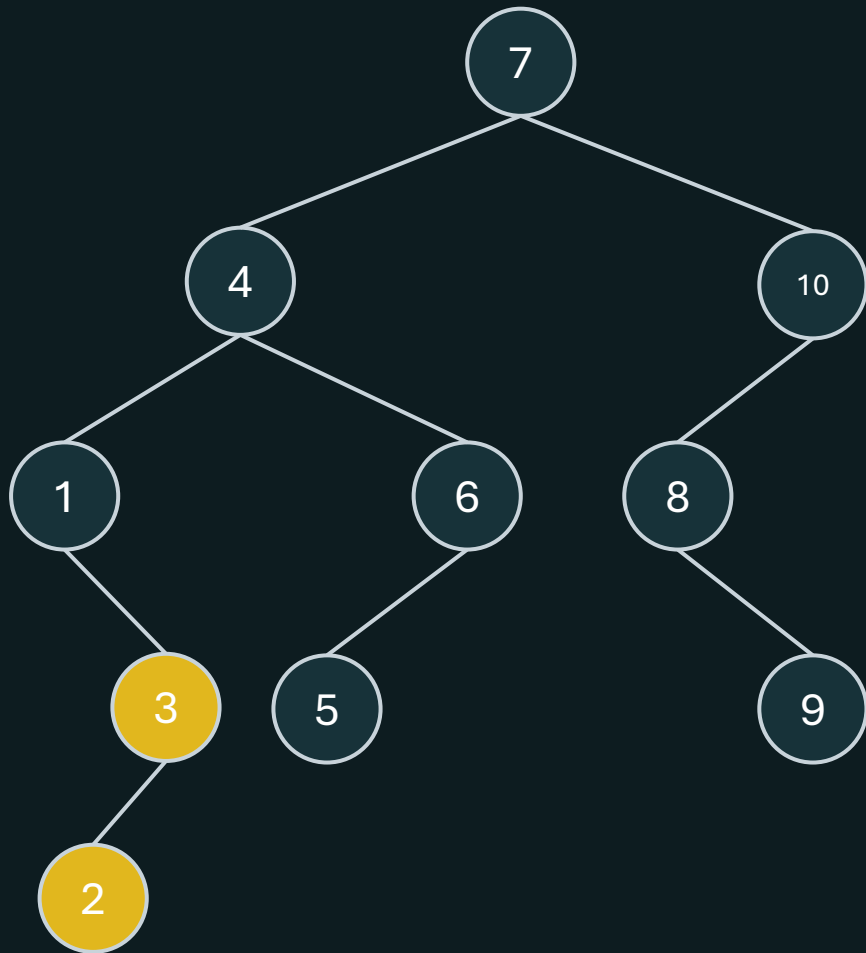
Procedura BVSMIN (minimum v BVS)

Vstup: Kořen BVS v

1. Pokud $\ell(v) = \emptyset$, vrátíme vrchol v .
2. Jinak vrátíme BVSMIN($\ell(v)$).

Výstup: Vrchol obsahující nejmenší klíč

Základní operace – Vlož prvek (*Insert*)



Procedura BVSINSERT (vkládání do BVS)

Vstup: Kořen BVS v , vkládaný klíč x

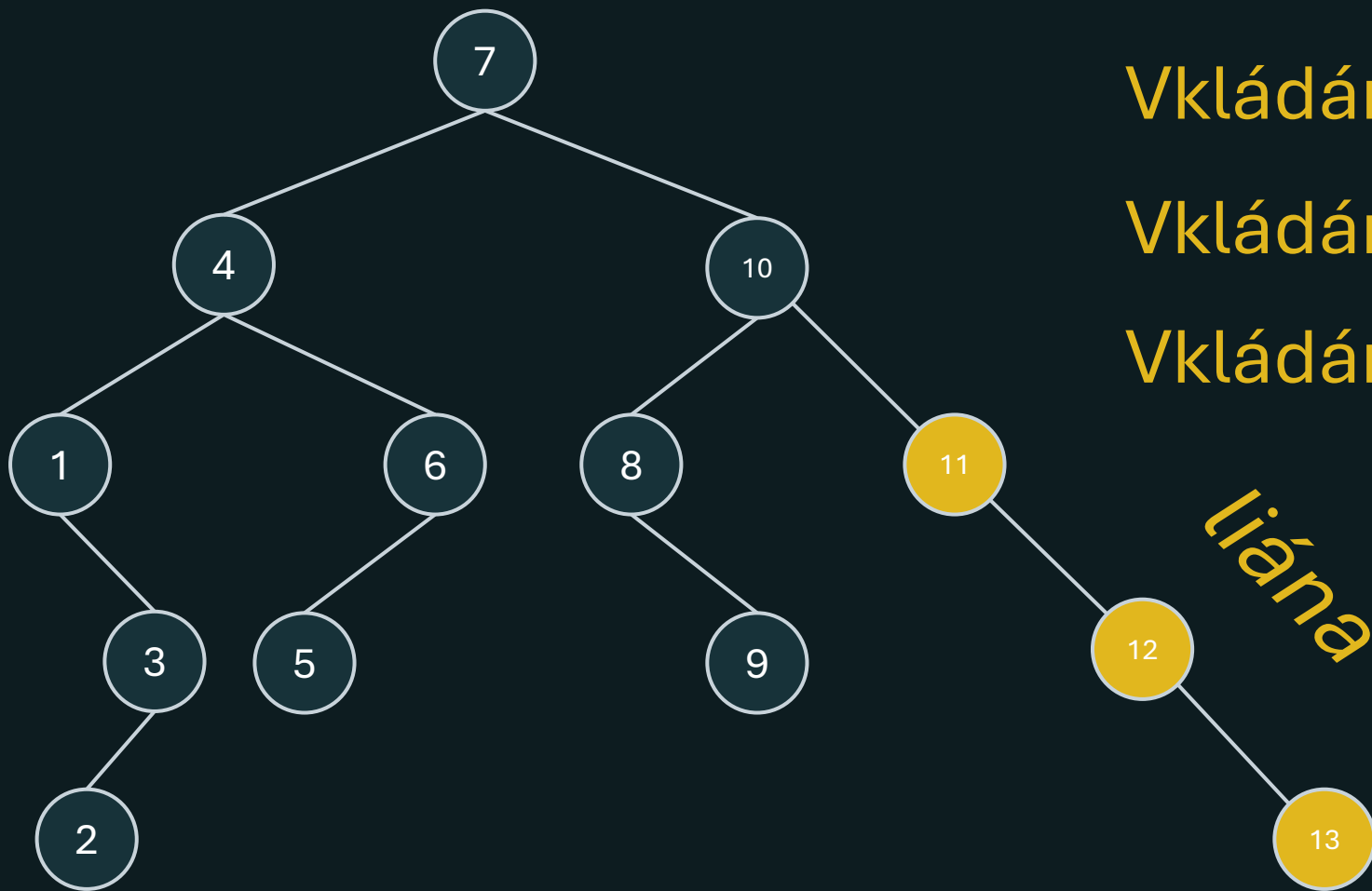
1. Pokud $v = \emptyset$, vytvoříme nový vrchol v s klíčem x a skončíme.
2. Pokud $x < k(v)$, položíme $\ell(v) \leftarrow \text{BVSINSERT}(\ell(v), x)$.
3. Pokud $x > k(v)$, položíme $r(v) \leftarrow \text{BVSINSERT}(r(v), x)$.
4. Pokud $x = k(v)$, klíč x se ve stromu již nachází a není třeba nic měnit.

Výstup: Nový kořen v

Vkládáme prvek s klíčem $x = 3$

Vkládáme prvek s klíčem $x = 2$

Základní operace – Vlož prvek (*Insert*)



Vkládáme prvek s klíčem $x = 11$

Vkládáme prvek s klíčem $x = 12$

Vkládáme prvek s klíčem $x = 13$

liána

- Tvoří hloubku stromu lineární a
tím i časovou složitost základních
operací

Základní operace – Smaž prvek (*Remove*)

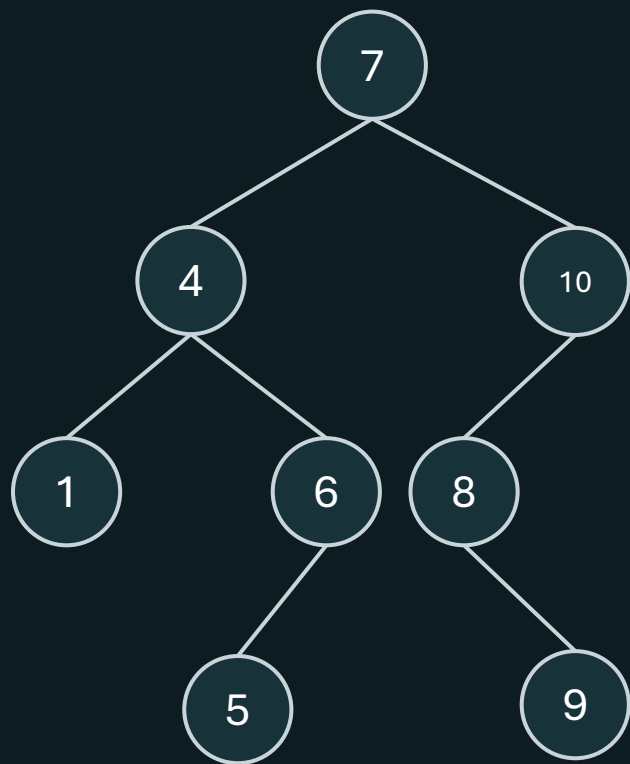
Procedura BVSDDELETE (mazání z BVS)

Vstup: Kořen BVS v , mazaný klíč x

1. Pokud $v = \emptyset$, vrátíme $\emptyset$. $\triangleleft$ klíč x ve stromu nebyl
2. Pokud $x < k(v)$, položíme $\ell(v) \leftarrow \text{BVSDDELETE}(\ell(v), x)$.
3. Pokud $x > k(v)$, položíme $r(v) \leftarrow \text{BVSDDELETE}(r(v), x)$.
4. Pokud $x = k(v)$: $\triangleleft$ chystáme se smazat vrchol v
5. Pokud $\ell(v) = r(v) = \emptyset$, vrátíme $\emptyset$. $\triangleleft$ byl to list
6. Pokud $\ell(v) = \emptyset$, vrátíme $r(v)$. $\triangleleft$ existoval jen pravý syn
7. Pokud $r(v) = \emptyset$, vrátíme $\ell(v)$. $\triangleleft$ existoval jen levý syn
8. $s \leftarrow \text{BVSMIN}(r(v))$ $\triangleleft$ máme oba syny: nahradíme následníka s
9. $k(v) \leftarrow k(s)$
10. $r(v) \leftarrow \text{BVSDDELETE}(r(v), k(s))$
11. Vrátíme v .

Výstup: Nový kořen v

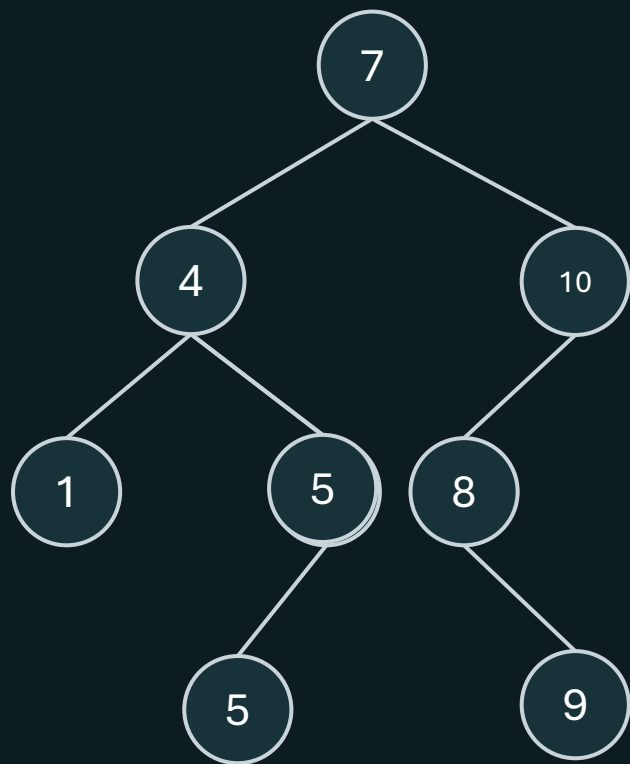
Základní operace – Smaž prvek (*Remove*)



Mažeme prvek s klíčem $x = 5$

Tento prvek byl *list*, tak jsme ho prostě odebrali

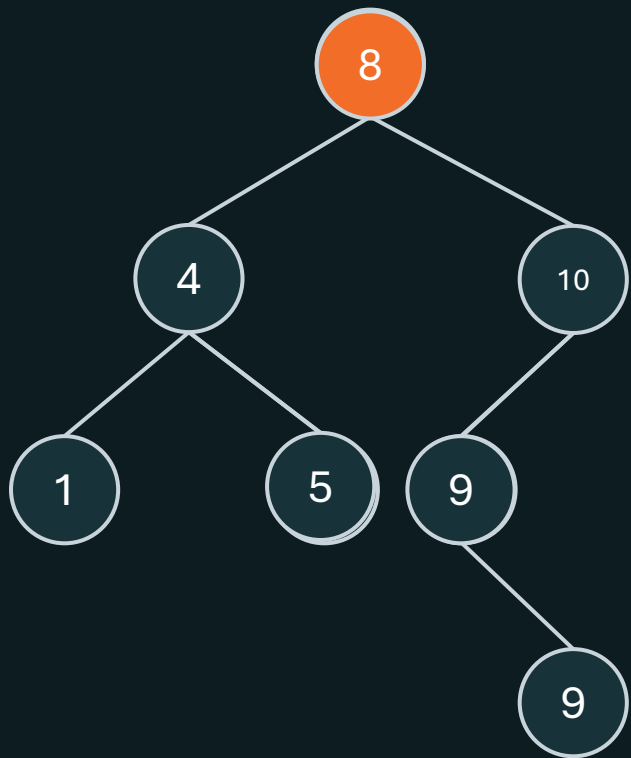
Základní operace – Smaž prvek (*Remove*)



Mažeme prvek s klíčem $x = 6$

Tento prvek měl jen *jednoho syna*,
takže jej nahradíme právě tímto synem

Základní operace – Smaž prvek (*Remove*)



Mažeme prvek s klíčem $x = 7$

Tento prvek má dva syny

Hledáme minimum (*FindMin*) v pravém podstromu

Klíč a hodnotu nalezeného minima uložíme do kořene

Z pravého podstromu odebereme prvek s klíčem předtím nalezeného minima



Vyváženost stromu

Časová složitost základních operací BVS

Vypiš strom (*Show*)

Najdi prvek (*Find*)

Najdi minimum (*FindMin*)

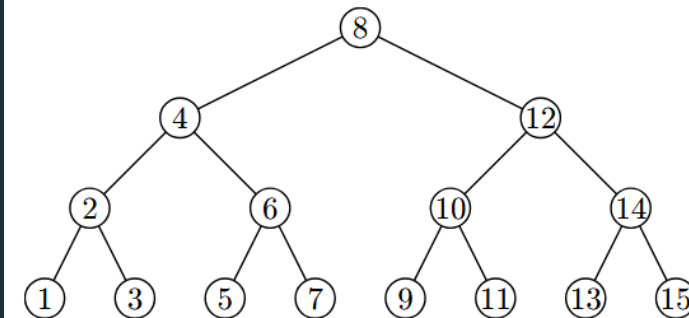
Vlož prvek (*Insert*)

Smaž prvek (*Remove*)

$O(n)$

$O(\text{hloubka stromu})$

Dokonale vyvážený strom



Binární vyhledávací strom nazveme *dokonale vyvážený*, pokud pro každý jeho vrchol v platí:

$$||L(v)| - |R(v)|| \leq 1$$

(počet vrcholů v levém podstromu v a v pravém podstromu v se liší nejvýše o 1)

- Pak by dokonale vyvážený BVS měl hloubku nejvýše $\log_2 n$
 - na kterékoliv cestě z kořene do listu klesá velikost podstromů s každým krokem alespoň dvakrát
- Ovšem kvůli udržování rovnováhy při opakovaném *Insert* i *Remove* stejně trvaly **lineárně dlouho**

Hloubkově vyvážený strom (AVL stromy)

- *AVL stromy*, neboť je vymysleli v roce 1962 ruští matematici Georgij Maximovič Aděľson-**V**eľskij a Jevgenij Michailovič **L**andis
- BVS nazveme *hloubkově vyvážený*, jestliže pro každý vrchol v platí:

$$|h(l(v)) - h(r(v))| \leq 1$$

- V každém vrcholu v si udržujeme hloubku jeho podstromu h
- Základní operace v $O(\log n)$ (kromě *Show*), ale komplikované

Hloubkově vyvážený strom (AVL stromy)

- BVS nazveme *hloubkově vyvážený*, jestliže pro každý vrchol v platí:

$$|h(l(v)) - h(r(v))| \leq 1$$

